Verdiqy ⚡

AI-Powered Competitive Programming & Codeforces Training Platform

Prakash Meena
GitHub: github.com/Pcmhacker-piro/Verdiqy
Live: verdiqy.vercel.app

1. The 30-Second Elevator Pitch (Start with this)

> *"Verdiqy is an AI-powered competitive programming platform designed around Codeforces. While competitive programmers have access to thousands of problems, they often plateau because they don't know what to practice next, or they get stuck on terse editorials. Verdiqy analyzes a user's submission history and topic weaknesses, recommends tailored practice sheets, and provides a multimodal AI mentor capable of reading problem screenshots, code snippets, and WA submissions to guide the candidate without spoiling solutions."*

2. Full-Stack System Architecture

React 19 | TanStack Start (SSR) | TypeScript 5.8 | Tailwind CSS v4 | Supabase (PostgreSQL) | Google Gemini 2.5 Flash | Codeforces REST API

Client & Presentation Layer:

- Full Server-Side Rendering (SSR) via **TanStack Start** for instant initial paint and SEO.
- Client-side caching and optimistic updates via **TanStack Query**.
- Accessible UI with **shadcn/ui**, Radix primitives, dark/light theme toggle, and universal `⌘K` command palette.

Server & Persistence Layer:

- Typed **Server Functions** (RPC) executed strictly on the edge/server runtime.
- Private credentials (`GEMINI_API_KEY` , `SUPABASE_SECRET_KEY`) kept strictly isolated server-side.
- Relational **PostgreSQL** with granular Row Level Security (RLS) policies.

3. The Backend Deep-Dive: Exactly What Lives There

🤖 AI Gateway & Multimodal Mentor

Server Only

Files: `ai-gateway.server.ts`, `mentor.functions.ts`

- Direct integration with **Google Gemini 2.5 Flash**.
- Multimodal pipeline: parses base64 problem diagrams, submission screenshots, and code attachments.
- System prompt enforces **Socratic algorithmic tutoring** (hints, edge case detection) rather than full solution dumping.

📊 Analytics & Topic Mastery Engine

Server Only

Files: `cf-dashboard.functions.ts`, `sync.server.ts`

- Computes per-tag mastery (accuracy %, average rating solved, tag frequency).
- Tracks mistake log: filters WA, TLE, MLE with re-attempt tracking.
- Contest analytics: calculates rating deltas and expected performance vs actual.

📄 Custom Practice Sheets Engine

RPC Functions

Files: `sheets.functions.ts`

- CRUD API for personalized and community practice sheets.
- Filter engine by rating bands (e.g. 1200–1600) and problem tags.
- Public sharing, cloning, problem ordering, and progress percentage.

🖼️ Codeforces Avatar Proxy Route

API Route

Files: `src/routes/api/public/cf-avatar.ts`

- Solves third-party avatar CORS failures & SSL hostname mismatches on Codeforces asset domains.
- Streams verified image buffers to client; falls back to SVG initials if Codeforces asset server is down.

4. Core Engineering Challenges & Solutions (Interview Gold)

1. Why TanStack Start Server Functions over Express/REST?

Challenge: Building a separate Express or NestJS backend introduces duplicate TypeScript interface maintenance, routing boilerplate, and synchronization overhead.
Solution: *"I implemented TanStack Start server functions (`createServerFn`). This provides end-to-end type inference from database queries straight to UI components, zero-overhead RPC routing, and guarantees that database secret keys never bundle into the browser."*

2. Handling Third-Party Codeforces API Constraints & Flakiness

Challenge: Codeforces rate limits requests (1 call every 2 seconds per IP) and their media CDN (`userpic.codeforces.org`) frequently goes offline independently of the main site.
Solution: *"I decoupled the client from direct asset requests. Avatars are proxied through an internal edge endpoint with upstream fallback. For data queries, TanStack Query server caching prevents redundant calls and protects users from hitting 429 rate limits."*

3. Multi-Tenant Database Security with PostgreSQL RLS

Challenge: Ensuring private user data (custom sheets, bug reports, AI conversations) is completely inaccessible to other users without relying solely on application-level `WHERE` clauses.
Solution: *"I enforced Row Level Security (RLS) directly in PostgreSQL. Even if an endpoint was misconfigured, the database engine itself rejects unauthorized reads/writes using `auth.uid() = user_id` checks."*

5. 3-Minute Live Demo Flow

1. Landing & Navigation
Show responsive design, dark/light theme, and command palette (`⌘K`).

2. Problems & Sheet Builder
Filter problems by tag/rating and build a custom curated sheet in 10 seconds.

3. AI Mentor & Analytics
Ask the mentor for intuition on a WA code snippet and show the tag mastery heatmap.

6. Tough Interview Questions & Perfect Answers

Q: "Did you just prompt an AI builder or use a boilerplate?"

Answer: *"I used modern AI tools to rapidly prototype the visual UI mockups. However, I engineered the full production architecture myself: decoupling the application into an independent TanStack Start SSR platform, designing the relational PostgreSQL schema with RLS, writing the Codeforces proxy, and building the server-side Gemini 2.5 Flash integration."*

Q: "Why PostgreSQL over MongoDB for competitive programming data?"

Answer: *"CP data is inherently relational: users own submissions, submissions link to problems, problems belong to contests and sheets. Relational schema with foreign keys and ACID constraints guarantees data integrity, which is much cleaner than denormalizing in a document store."*

Q: "How do you ensure the AI mentor doesn't spoil the answer immediately?"

Answer: *"I created a Socratic coaching system prompt that classifies user queries into 'hint', 'edge-case', or 'conceptual'. It provides time/space complexity guidance and thought questions first, and only provides concrete code when explicitly requested."*